

Case Study

Problem Statement : Predicting Delivery Risk at Order Time - Right now, the business only learns an order will be late after the fact, when a customer complains, or the delivery is already overdue. Operations wants to move from reactive to proactive: they'd like to know, the moment an order is placed, whether it carries meaningful risk of arriving late, so they can intervene early, expedite it, notify the customer ahead of time, or flag it for manual review.

→ How you defined "late," and exactly what your prediction target is?

The dataset already had a business-defined label, `Late_delivery_risk`, which was verified 1:1 encoding of Delivery Status (1 = "Late delivery", 0 = "Advance shipping"/"Shipping on time"). Used this, since it reflects what the business already tracks.

One important data-quality catch: 2,514 orders had Delivery Status = "Shipping canceled" and were labeled 0 in the raw data but these orders never actually shipped, so "late" is undefined for them, not "false". Dropped these rows entirely rather than let the model learn that cancellation patterns look like on-time delivery.

→ Which data to use as features, and why?

1. Leakage (excluded): actual shipping days, Delivery Status, actual ship date - these all only knowable after the "late" has occurred
2. PII/free text (excluded): emails, names, passwords, product descriptions/images
3. Ambiguous availability (excluded): Order Status (changes after order placement), profit/benefit fields (deemed conceptually unrelated to shipping timing)
4. Zero-variance (auto-excluded): Product Status was constant across every single row carried literally zero information

7 features which were used - Category Name, Order Region, Type (categorical);

Days for shipment (scheduled), `order_dow`, `order_month`, `order_hour` (numeric).

These were kept because it supports most of our hypothesis. Some regions have different effects on delivery like, Canada has less risk of being late whereas Asia has more risk of the delivery being late.

Date and time is very important as few seasons and holidays are taken in account for few regions, if the order is made on weekend or on a christmas holidays, it is going to be delayed (more no. of orders to deliver, during holidays less no. of people are working).

→ Your model choice and the reasoning behind it ?

Every model is using sklearn-

- DummyClassifier (majority-class) - the absolute floor any model must beat
- Logistic Regression - fast to train, and easy to find linear relationships. It's the right first real model because it gives you a transparent baseline to measure any added complexity against.
- Random Forest - the month × region seasonality pattern I tested directly earlier, where different markets swing by very different amounts across the year. Random Forest can capture exactly that kind of interaction, so trying it was a targeted test of that specific hypothesis

→ How you split your data for training/testing, and why you split it that way ?

Used a time-based split (train on orders before 2017-05-19, test on everything after) rather than random splitting, since the model will always be trained on past orders and scored on new ones in production, a random split would leak future patterns into training.

→ A simple baseline you compared your model against, and how your model performed relative to it

- Dummy Classifier-Accuracy 0.579, no real predictive signal.
- Logistic Regression(baseline - Scheduled Days alone) - used this because column itself contributes to 87% in the model, using this as baseline and checking if adding new features will help the model at all.
- Logistic Regression(using all the 7 feature)
- RandomForest(all 7 feature)

RESULTS							
	Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
	Dummy (majority class)	0.579	0.579	1.000	0.733	NaN	NaN
	Baseline: Scheduled Days alone	0.711	0.840	0.619	0.713	0.730	0.739
	Logistic Regression (full features)	0.710	0.836	0.622	0.713	0.733	0.777
	Random Forest (full features)	0.718	0.852	0.621	0.718	0.758	0.837
OVERFITTING CHECK (train vs test)							
	Model	Set	Accuracy	F1	ROC-AUC		
	Logistic Regression	TRAIN	0.703	0.712	0.723		
	Logistic Regression	TEST	0.710	0.713	0.733		
	Random Forest	TRAIN	0.720	0.725	0.800		
	Random Forest	TEST	0.718	0.718	0.758		

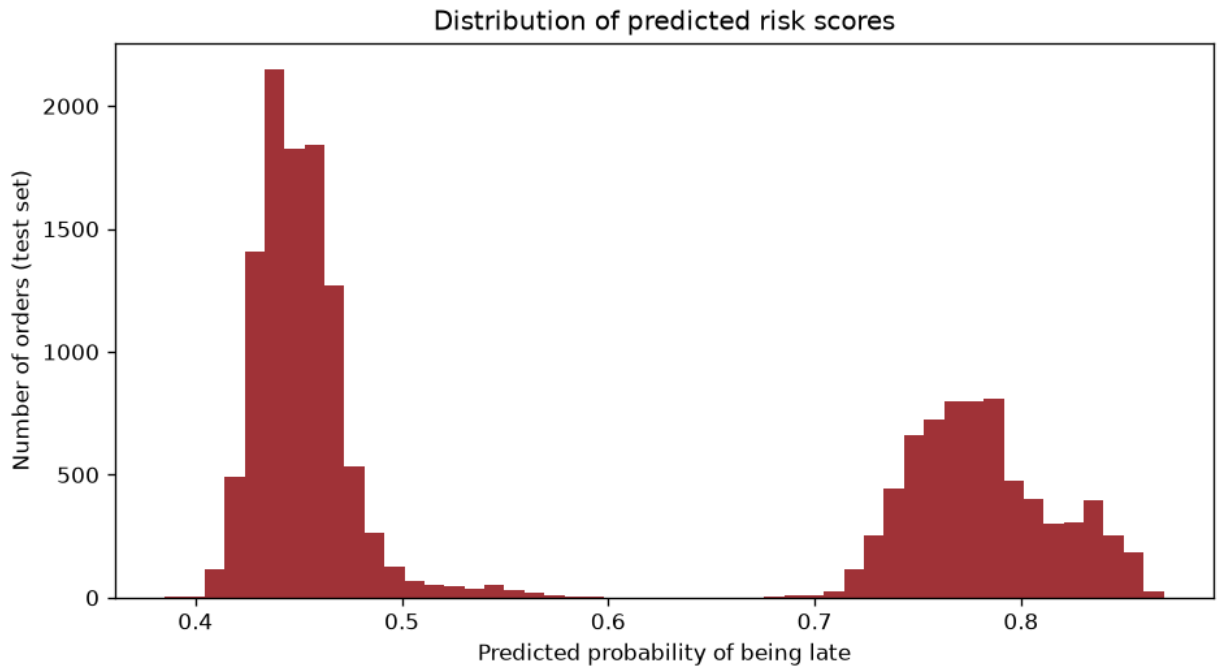
Random Forest is the only model that meaningfully beats the LogisticR baseline (+0.028 ROC-AUC). Precision stays consistently high (~84–85%) across every real model which means when the model flags an order, it's usually right, which matters in quality of output.

→ What the output would look like in production, and how operations would actually use it? Initially the idea was to predict if they have any delivery risk, 0 or 1. But that would cause a lot of bulk orders to check manually. Upon further research I came across the idea of dividing the risk in 3 levels - low level, medium level and high level.

This would reduce manual work to only a high level.

To divide them into 3 different bins I used the probability of the prediction being right. I plotted a graph to see where the threshold lands naturally (the gaps between the graph).

It looked something like this :-



Orders cluster into two bands with almost nothing in between (roughly 0.40- 0.55, 0.70 - 0.85 most of the data lies), because one feature (Scheduled Days) dominates the model. I used the precision/recall table to pick the second boundary at the point where precision first reaches 90% because that makes it certain that it will be late and the probability of it being false is less. This also makes sure manual work is done only on these orders and rest can be handled with automated workflow (example medium risk - automated message).

This is how the delivery risk column was created which would help ops team directly in managing such situations.

→ Key risks or failure modes of your approach, and how you'd guard against them

- Over-reliance on one feature. Days for shipment (scheduled) is approx 85% of the model's decision-making. This might cause the model to break down without any obvious logic error. We could distribute the reliance on multiple features which will push the model to find more patterns and make it more accurate for the long term.

- The model misses roughly 38% of genuinely late orders (recall 0.62). Every miss is a customer surprised with zero warning. If missed late orders are costly we might need to lower the threshold and accept more false alarms.
- The model might fail over time if not maintained and updated with the recent activity and new patterns. Regularly maintaining the model would help us minimize the error rate by the model.